

1) Introduction to APIs

Q1) What is an API (Application Programming Interface)? **API**

(Application Programming Interface)

An API is a set of rules that allows one software application to communicate with another software application.

It helps applications share data and services easily and quickly.

Examples:

- Weather apps
- Online payment apps
- Login systems
- Google Maps

Advantages of API:

1. Easy data sharing
2. Fast communication
3. Saves development time
4. Secure connection
5. Easy to use

Conclusion:

API helps different applications work together and exchange information easily.

Q2) Types of APIs: REST, SOAP.

Types of APIs: REST and SOAP

1. REST API:

REST stands for Representational State Transfer. It is a simple and fast API used in web and mobile applications. REST API uses HTTP methods like GET, POST, PUT, and DELETE. It mainly uses JSON format for data sharing.

Features of REST API:

- Fast and lightweight
- Easy to use
- Mostly used in web applications

Examples:

- Weather apps
- Shopping websites • Mobile applications

2. SOAP API:

SOAP stands for Simple Object Access Protocol. It is a secure API used for communication between applications. SOAP API uses XML format for data transfer and follows strict rules.

Features of SOAP API:

- More secure
- Reliable communication
- Used in banking and enterprise systems

Examples:

- Banking systems
- Online payment systems
- Railway reservation systems

Difference Between REST and SOAP:

- REST is simple and fast, while SOAP is more secure.
- REST uses JSON, while SOAP uses XML.
- REST is lightweight, while SOAP is heavyweight.

Conclusion:

REST and SOAP are important types of APIs. REST is easy and fast, while SOAP provides better security and reliability.

Q3) Why are APIs important in web development?

APIs are important in web development because they help different applications communicate and share data with each other. APIs make web applications faster, easier, and more powerful.

Importance of APIs in Web Development:

1. Data Sharing:
APIs allow applications to exchange information easily between servers, websites, and mobile apps.
2. Saves Development Time:
Developers can use existing APIs instead of creating everything from scratch.
3. Easy Integration:
APIs help connect services like payment systems, maps, login systems, and weather data into websites.
4. Better User Experience:
APIs provide fast and smooth communication, which improves website performance and user experience.
5. Automation:
APIs help automate tasks such as sending messages, processing payments, and updating data.
6. Security:

APIs allow secure communication between applications using authentication and authorization methods.

Examples:

- Online payment gateways
- Google Maps integration
- Social media login
- Weather applications

Conclusion:

APIs are an important part of web development because they help applications connect, share data, and provide better services to users.

2. Requirements for Web Development

Projects Q1) Understanding project requirements.

Understanding project requirements means knowing what the project needs, what features should be included, and how the project should work before development starts.

It helps developers understand:

- What the client wants
- What problem the project will solve
- Which features are needed
- How the system should work

Steps in Understanding Project Requirements:

1. Collect information from the client
2. Identify project goals
3. Understand user needs
4. List required features
5. Plan project structure and design

Importance of Understanding Project Requirements:

- Reduces mistakes
- Saves development time
- Improves project quality
- Helps complete the project successfully
- Makes communication clear between client and developer

Example:

If a client wants an online shopping website, the requirements may include:

- User login
- Product listing
- Add to cart
- Online payment
- Order tracking

Conclusion:

Understanding project requirements is the first and most important step in software and web development because it helps create the correct and successful project.

Q2) Setting up the environment and installing necessary packages.

Setting up the environment means preparing the computer and software tools needed for project development. Installing necessary packages means adding required libraries and software to run the project properly.

Steps for Setting Up the Environment:

1. Install the programming language such as Python.
2. Install the code editor such as VS Code.
3. Create a project folder.
4. Create a virtual environment.
5. Install required packages and libraries.

Installing Necessary Packages:

Packages are additional tools or libraries used in the project development.

Example commands in Python:

- Install Django: `pip install django`
- Install Flask:
`pip install flask`
- Install requests package:
`pip install requests`

Importance:

- Helps run the project correctly
- Manages project dependencies
- Makes development easier and faster
- Avoids software conflicts

Conclusion:

Setting up the environment and installing necessary packages are important steps because they prepare the system for smooth and successful project development.

3. Serialization in Django REST Framework Q1)

What is Serialization?

Serialization is the process of converting data or objects into a format that can be stored or sent easily.

In easy words, serialization means changing complex data into a simple format like JSON or XML.

Why Serialization is Used:

- To send data between applications
- To save data in files or databases
- To transfer data through APIs

Example:

Python object → JSON format

Example Data:

Name = "Ashish"

Age = 20

After Serialization:

```
{  
  "name": "Ashish",  
  "age": 20  
}
```

Importance of Serialization:

- Easy data sharing
- Used in APIs
- Helps in data storage
- Fast communication between systems

Conclusion:

Serialization converts data into a simple format so that applications can store, share, and transfer data easily.

Q2) Converting Django QuerySets to JSON.

Converting Django QuerySets to JSON means changing Django database data into JSON format so it can be used in APIs and web applications.

Example: models.py

```
from django.db import models
```

```
class Student(models.Model):    name =  
models.CharField(max_length=100)    age =  
models.IntegerField()
```

views.py

```
from django.http import JsonResponse from  
.models import Student
```

```
def student_data(request):    students =  
list(Student.objects.values())    return  
JsonResponse(students, safe=False)
```

Explanation:

- Student.objects.values() gets all database records.
- list() converts QuerySet into a list.
- JsonResponse() converts data into JSON format.

Output:

```
[  
  {  
    "id": 1,  
    "name": "Ashish",  
    "age": 20  
  },  
  {  
    "id": 2,  
    "name": "Satyam",  
    "age": 22  
  }  
]
```

Importance:

- Used in REST APIs
- Easy data sharing
- Helps frontend and backend communication

Conclusion:

Django QuerySets are converted into JSON to send database data easily between applications and APIs.

Q3) Using serializers in Django REST Framework (DRF).

Using serializers in Django REST Framework (DRF) means converting Django model data into JSON format and converting JSON data back into Django objects.

Serializers help in:

- Data conversion

- Data validation
- Sending API responses

Example: models.py

```
from django.db import models
```

```
class Student(models.Model):
    name = models.CharField(max_length=100)
    age = models.IntegerField()
serializers.py
```

```
from rest_framework import serializers
from .models import Student
```

```
class StudentSerializer(serializers.ModelSerializer):
    class Meta:
        model = Student
        fields = '__all__'
```

views.py

```
from rest_framework.response import Response
from rest_framework.views import APIView
from .models import Student
from .serializers import StudentSerializer
```

```
class StudentAPIView(APIView):
```

```
    def get(self, request):
        students = Student.objects.all()
        serializer = StudentSerializer(students, many=True)
```

```
    return Response(serializer.data) Output:
```

```
[
  {
    "id": 1,
    "name": "Ashish",
    "age": 20
  }
]
```

Importance of Serializers:

- Converts model data into JSON
- Validates input data
- Easy API development
- Secure data handling

Conclusion:

Serializers in DRF are used to convert Django model data into JSON format and make API development easier and faster.

4. Requests and Responses in Django REST Framework Q1)

HTTP request methods (GET, POST, PUT, DELETE).

HTTP request methods are used to communicate between the client and the server in web applications and APIs.

Main HTTP Request Methods:

1. GET:

GET method is used to get or fetch data from the server.

Example:

- Show student data
- Fetch product list

Example URL:

GET /students

2. POST:

POST method is used to send or add new data to the server.

Example:

- Add new student
- Create new product

Example:

POST /students

3. PUT:

PUT method is used to update existing data on the server.

Example:

- Update student information
- Edit product details

Example:

PUT /students/1

4. DELETE:

DELETE method is used to remove data from the server.

Example:

- Delete student record
- Remove product

Example:

DELETE /students/1

Difference Between Methods:

Method	Purpose
GET	Fetch data
POST	Add new data
PUT	Update data
DELETE	Remove data

Conclusion:

HTTP request methods help APIs and web applications perform operations like fetching, adding, updating, and deleting data.

Q2) Sending and receiving responses in DRF.

Sending and receiving responses in Django REST Framework (DRF) means handling client requests and returning data in JSON format using APIs.

In DRF:

- Client sends a request to the server
- Server processes the request
- Server sends a response back to the client

Example: views.py from

```
rest_framework.views import APIView from
```

```
rest_framework.response import
```

```
Response
```

```
class StudentAPIView(APIView):
```

```
    def get(self, request):
data = {
    "name": "Ashish",
    "age": 20
}
    return Response(data) Explanation:
```

- APIView is used to create API views.
- request receives client data.
- Response() sends data back in JSON format.

Output:

```
{
  "name": "Ashish",
  "age": 20
}
```

Receiving Data Example:

```
def post(self, request):
    print(request.data)
    return Response({"message": "Data Received"})
```

Explanation:

- request.data receives data sent by the client.
- Response() returns a response message.

Importance:

- Used in API communication
- Sends JSON responses
- Handles client requests easily
- Makes API development simple

Conclusion:

DRF helps developers easily receive client requests and send JSON responses in web APIs.

5. Views in Django REST Framework

Q1) Understanding views in DRF: Function-based views vs Class-based views.

Understanding views in Django REST Framework (DRF) means learning how APIs handle requests and send responses.

There are two main types of views in DRF:

1. Function-Based Views (FBV)
2. Class-Based Views (CBV)
3. Function-Based Views (FBV):
Function-based views use Python functions to handle requests.

Example:

```
from rest_framework.decorators import api_view from
rest_framework.response import Response
```

```
@api_view(['GET']) def student_view(request):
return Response({"message": "Hello Student"})
```

Features:

- Simple and easy to understand
- Good for small projects
- Uses functions

Advantages:

- Easy syntax
- Fast to write
- Good for beginners

Disadvantages:

- Difficult to manage large projects
- Code repetition may happen

2. Class-Based Views (CBV):

Class-based views use Python classes to handle requests.

Example:

```
from rest_framework.views import APIView
from rest_framework.response import Response
```

```
class StudentAPIView(APIView):
```

```
    def get(self, request):        return
Response({"message": "Hello Student"})
```

Features:

- Uses classes and methods
- Better code organization
- Good for large projects

Advantages:

- Reusable code
- Better project structure
- Easy to extend

Disadvantages:

- Slightly complex for beginners

Difference Between FBV and CBV:

Function-Based Views	Class-Based Views
Uses functions	Uses classes
Simple and easy	More organized
Good for small projects	Good for large projects
Less reusable	More reusable

Conclusion:

Function-based views are simple and beginner-friendly, while class-based views are more powerful and better for large and professional DRF projects.

6. URL Routing in Django REST Framework

Q1) Defining URLs and linking them to views.

Defining URLs and linking them to views means connecting website URLs with Python view functions or classes in Django.

When a user opens a URL, Django sends the request to the correct view.

Example: views.py

```
from django.http import HttpResponse

def home(request):    return
    HttpResponse("Welcome Home") urls.py

from django.urls import path

from .views import home

urlpatterns = [
    path("", home),
]
```

Explanation:

- path() defines the URL.
- home is the view connected to the URL.
- When the user opens the URL, the home view runs.

Class-Based View Example:

views.py

```
from rest_framework.views import APIView
```

```

from rest_framework.response import Response

class StudentAPIView(APIView):

    def get(self, request):
        return
        Response({"message": "Student API"}) urls.py

from django.urls import path
from .views import StudentAPIView

urlpatterns = [
    path('student/', StudentAPIView.as_view()),
]

```

Explanation:

- `as_view()` connects class-based views to URLs.
- Opening `/student/` runs the API view.

Importance:

- Connects pages and APIs
- Handles user requests
- Organizes project routing
- Important part of Django applications

Conclusion:

Defining URLs and linking them to views helps Django know which function or class should run for a specific URL request.

7. Pagination in Django REST Framework

Q1) Adding pagination to APIs to handle large data sets.

Adding pagination to APIs means dividing large amounts of data into smaller pages so the API can send data step by step instead of sending all records at one time.

Pagination helps improve:

- Performance
- Speed
- Data management

Example:

Instead of showing 1000 records together, pagination shows:

- Page 1 → 10 records
- Page 2 → next 10 records DRF Pagination Example:

settings.py

```
REST_FRAMEWORK = {
    'DEFAULT_PAGINATION_CLASS': 'rest_framework.pagination.PageNumberPagination',
    'PAGE_SIZE': 5
}
```

views.py

```
from rest_framework.generics import ListAPIView
from .models import Student
from .serializers import StudentSerializer
```

```
class StudentListAPIView(ListAPIView):
    queryset = Student.objects.all()
    serializer_class = StudentSerializer
```

Output:

```
{
  "count": 20,
  "next": "http://127.0.0.1:8000/student/?page=2",
  "previous": null,
  "results": [
    {
      "id": 1,
      "name": "Ashish"
    }
  ]
}
```

Advantages of Pagination:

- Faster API response
- Handles large data easily
- Reduces server load
- Better user experience

Types of Pagination in DRF:

1. Page Number Pagination
2. Limit Offset Pagination
3. Cursor Pagination

Conclusion:

Pagination in APIs helps manage large datasets by sending data in small pages, improving performance and user experience.

8. Settings Configuration in Django

Q1) Configuring Django settings for database, static files, and API keys.

Configuring Django settings means setting up important project configurations like database connection, static files, and API keys inside the settings.py file.

1. Database Configuration:

Database settings are used to connect Django with a database like SQLite or MySQL.

Example:

```
DATABASES = {  
    'default': {  
        'ENGINE': 'django.db.backends.sqlite3',  
        'NAME': BASE_DIR / 'db.sqlite3',  
    }  
}
```

MySQL Example:

```
DATABASES = {  
    'default': {  
        'ENGINE': 'django.db.backends.mysql',  
        'NAME': 'mydatabase',  
        'USER': 'root',  
        'PASSWORD': '1234',  
        'HOST': 'localhost',  
        'PORT': '3306',  
    }  
}
```

2. Static Files Configuration:

Static files include CSS, JavaScript, and images.

Example:

```
STATIC_URL = 'static/'
```

```
STATICFILES_DIRS = [  
    BASE_DIR / "static",  
]
```

3. API Key Configuration:

API keys are used to connect external services securely.

Example:

```
API_KEY = "your_api_key_here"
```

Using Environment Variables:

```
import os
```

```
API_KEY = os.getenv("API_KEY") Importance:
```

- Connects the database
- Manages static files
- Secures API credentials
- Helps run the project properly

Conclusion:

Configuring Django settings is important for database connection, static file handling, and secure API integration in Django projects.

Configuring Django settings means setting up important project configurations like database connection, static files, and API keys inside the settings.py file.

1. Database Configuration:

Database settings are used to connect Django with a database like SQLite or MySQL.

Example:

```
DATABASES = {
    'default': {
        'ENGINE': 'django.db.backends.sqlite3',
        'NAME': BASE_DIR / 'db.sqlite3',
    }
}
```

MySQL Example:

```
DATABASES = {
    'default': {
        'ENGINE': 'django.db.backends.mysql',
        'NAME': 'mydatabase',
        'USER': 'root',
        'PASSWORD': '1234',
        'HOST': 'localhost',
        'PORT': '3306',
    }
}
```

2. Static Files Configuration:

Static files include CSS, JavaScript, and images.

Example:

```
STATIC_URL = 'static/'

STATICFILES_DIRS = [
    BASE_DIR / "static",
]
```

3. API Key Configuration:

API keys are used to connect external services securely.

Example:

```
API_KEY = "your_api_key_here"
```

Using Environment Variables:

```
import os
```

API_KEY = os.getenv("API_KEY") Importance:

- Connects the database
- Manages static files
- Secures API credentials
- Helps run the project properly

Conclusion:

Configuring Django settings is important for database connection, static file handling, and secure API integration in Django projects.

9. Project Setup

Q1) Setting up a Django REST Framework project.

Setting up a Django REST Framework (DRF) project means creating a Django project and installing DRF to build REST APIs.

Steps to Set Up a DRF Project:

1. Install Django:

```
pip install django
```

2. Install Django REST Framework:

```
pip install djangorestframework
```

3. Create Django Project:

```
django-admin startproject myproject
```

4. Go to Project Folder: `cd myproject`

5. Create Application:

```
python manage.py startapp myapp
```

6. Add Apps in settings.py:

```
INSTALLED_APPS = [  
    'django.contrib.admin',  
    'django.contrib.auth',  
    'django.contrib.contenttypes',  
    'django.contrib.sessions',  
    'django.contrib.messages',  
    'django.contrib.staticfiles',
```

```
'rest_framework',  
'myapp',  
]
```

7. Create API View:

views.py

```
from rest_framework.views import APIView  
from rest_framework.response import Response
```

```
class HomeAPIView(APIView):  
  
    def get(self, request):  
        return  
        Response({"message": "Welcome to DRF"})
```

8. Create URLs: urls.py

```
from django.urls import path  
from .views import HomeAPIView  
  
urlpatterns = [  
    path('home/', HomeAPIView.as_view()),  
]
```

Project urls.py

```
from django.contrib import admin  
from django.urls import path, include  
urlpatterns = [  
    path('admin/',  
        admin.site.urls),  
    path("", include('myapp.urls')),  
]
```

9. Run Server:

```
python manage.py runserver
```

Output: <http://127.0.0.1:8000/home/>

Advantages:

- Easy API development
- JSON response support
- Fast backend development
- Secure API handling

Conclusion:

Setting up a DRF project helps developers create REST APIs easily using Django and Django REST Framework.

10. Social Authentication, Email, and OTP Sending API

Q1) Implementing social authentication (e.g., Google, Facebook) in Django.

Implementing social authentication in Django means allowing users to log in using social accounts like Google or Facebook instead of creating a new username and password.

Social authentication uses OAuth2 for secure login.

Advantages:

- Easy login process
- Faster registration
- Secure authentication
- Better user experience

Steps to Implement Social Authentication in Django:

1. Install Required Packages:

```
pip install social-auth-app-django
```

2. Add Apps in settings.py:

```
INSTALLED_APPS = [  
    'django.contrib.admin',  
    'django.contrib.auth',  
    'django.contrib.contenttypes',  
    'django.contrib.sessions',  
    'django.contrib.messages',  
    'django.contrib.staticfiles',  
  
    'social_django',  
]
```

3. Add Authentication Backends:

```
AUTHENTICATION_BACKENDS = (  
    'social_core.backends.google.GoogleOAuth2',  
    'social_core.backends.facebook.FacebookOAuth2',  
  
    'django.contrib.auth.backends.ModelBackend',  
)
```

4. Add Google API Keys:

```
SOCIAL_AUTH_GOOGLE_OAUTH2_KEY = 'your_google_client_id'  
SOCIAL_AUTH_GOOGLE_OAUTH2_SECRET = 'your_google_secret_key'
```

5. Add Facebook API Keys:

```
SOCIAL_AUTH_FACEBOOK_KEY = 'your_facebook_app_id' SOCIAL_AUTH_FACEBOOK_SECRET  
= 'your_facebook_secret_key'
```

6. Add URLs:

```
from django.urls import path, include
```

```
urlpatterns = [  
    path('social-auth/', include('social_django.urls', namespace='social')),  
]
```

7. Login URL Example:

<http://127.0.0.1:8000/social-auth/login/google-oauth2/> How

It Works:

- User clicks Google or Facebook login
- Django redirects to the social platform
- User gives permission
- Django receives user information
- User logs into the website

Conclusion:

Social authentication in Django helps users log in quickly and securely using Google or Facebook accounts.

Q2) Sending emails and OTPs using third-party APIs like Twilio, SendGrid.

Sending emails and OTPs using third-party APIs means using external services like Twilio and SendGrid to send verification codes, notifications, and emails from a Django application.

1. Twilio for OTP Sending:

Twilio is used to send SMS and OTP messages to mobile numbers.

Install Twilio:

```
pip install twilio
```

Example:

```
from twilio.rest import Client
```

```
account_sid = 'your_account_sid'  
auth_token = 'your_auth_token'
```

```
client = Client(account_sid, auth_token)
```

```
message = client.messages.create(  
    body='Your OTP is 1234',  
    from_='+1234567890',  
    to='+919999999999'  
)
```

```
print(message.sid)
```

2. SendGrid for Sending Emails:

SendGrid is used to send emails from applications.

Install SendGrid:

```
pip install sendgrid
```

Example:

```
from sendgrid import SendGridAPIClient  
from sendgrid.helpers.mail import Mail
```

```
message = Mail(  
    from_email='test@example.com',  
    to_emails='user@example.com',    subject='OTP  
Verification',  
    html_content='<strong>Your OTP is 1234</strong>'  
)
```

```
sg = SendGridAPIClient('your_api_key') response  
= sg.send(message)
```

```
print(response.status_code) Advantages:
```

- Fast email and SMS delivery
- Secure OTP verification
- Easy integration with Django
- Improves user authentication

Uses:

- User registration
- Password reset
- Two-factor authentication
- Notifications

Conclusion:

Third-party APIs like Twilio and SendGrid help Django applications send emails and OTPs securely and efficiently.

11. RESTful API Design

Q1) REST principles: statelessness, resource-based URLs, and using HTTP methods for CRUD operations.

REST principles are the basic rules followed in REST APIs to make communication simple, fast, and organized.

Main REST Principles:

1. Statelessness:

In REST API, every request is independent. The server does not remember previous requests.

This means:

- Each request contains complete information
- Server does not store client session data

Example:

If a user sends a request, authentication data must be included every time.

Advantages:

- Better performance
- Easy scalability
- Simple server management

2. Resource-Based URLs:

In REST, everything is treated as a resource, and each resource has a unique URL.

Example:

/students/
/products/ /orders/

Here:

- students is a resource
- products is a resource

Good REST URL Example:

/students/1

This URL accesses student data with ID 1.

Advantages:

- Easy to understand
- Organized API structure

- Better readability

3. Using HTTP Methods for CRUD Operations:

REST APIs use HTTP methods to perform CRUD operations.

CRUD Operations:

- Create
 - Read
 - Update
 - Delete
- HTTP Methods:

HTTP Method	CRUD Operation	Purpose
GET	Read	Fetch data
POST	Create	Add new data
PUT	Update	Modify existing data
DELETE	Delete	Remove data

Examples:

GET /students

POST /students

PUT /students/1

DELETE /students/1

Conclusion:

REST principles help create simple, scalable, and organized APIs using stateless communication, resource-based URLs, and HTTP methods for CRUD operations.

12. CRUD API (Create, Read, Update, Delete)

Q1) What is CRUD, and why is it fundamental to backend development?

CRUD stands for Create, Read, Update, and Delete. These are the four basic operations used in backend development and databases.

CRUD Operations:

1. Create:

Used to add new data to the database.

Example:

- Add new student
- Register new user

HTTP Method:

POST

2. Read:
- Used to get or fetch data from the database.

Example:

- Show product list
- View user details

HTTP Method:

GET

3. Update:
- Used to modify existing data.

Example:

- Update profile
- Edit product details

HTTP Method:

PUT

4. Delete:
- Used to remove data from the database.

Example:

- Delete user
- Remove product

HTTP Method:

DELETE

Why CRUD is Important in Backend Development:

- Manages database operations
- Helps create dynamic applications
- Used in APIs and web applications
- Handles user data efficiently
- Core functionality of backend systems

Example in Real Life:

- Social media apps create posts
- Users read posts
- Users update profiles
- Users delete comments

Conclusion:

CRUD is fundamental to backend development because it handles all important database operations like creating, reading, updating, and deleting data.

13. Authentication and Authorization API

Q1) Difference between authentication and authorization.

Authentication	Authorization
Authentication means verifying the identity of the user.	Authorization means checking what the user is allowed to access.
It answers: “Who are you?”	It answers: “What can you do?”
Happens before authorization.	Happens after authentication.
Used for login verification.	Used for permission checking.
Requires username and password.	Requires roles and permissions.
Example: User login with email and password.	Example: Admin can delete data, normal user cannot.
Main purpose is security verification.	Main purpose is access control.

Conclusion:

Authentication checks the user identity, while authorization checks user permissions and access rights.

Q2) Implementing authentication using Django REST Framework’s token-based system.

Implementing authentication using Django REST Framework’s token-based system means verifying users using a unique token instead of checking username and password every time.

After login, the server gives a token to the user. The user sends this token with every request for authentication.

Advantages:

- Secure authentication
- Easy API access
- No need to send password repeatedly
- Commonly used in REST APIs

Steps to Implement Token Authentication in DRF:

1. Install DRF:

```
pip install djangorestframework
```

2. Add Apps in settings.py:

```
INSTALLED_APPS = [  
    'rest_framework',  
    'rest_framework.authtoken',  
]
```

3. Run Migration: `python manage.py migrate`

4. Configure Authentication:

```
REST_FRAMEWORK = {  
    'DEFAULT_AUTHENTICATION_CLASSES': [  
        'rest_framework.authentication.TokenAuthentication',  
    ]  
}
```

5. Create Token Automatically:

signals.py

```
from django.conf import settings  
from django.db.models.signals import post_save  
from django.dispatch import receiver  
from rest_framework.authtoken.models import Token
```

```
@receiver(post_save, sender=settings.AUTH_USER_MODEL) def  
create_auth_token(sender, instance=None, created=False, **kwargs):  
    if created:  
        Token.objects.create(user=instance)
```

6. Add Token URL:

urls.py

```
from django.urls import path
```

```
from rest_framework.authtoken.views import obtain_auth_token
```

```
urlpatterns = [  
    path('api-token-auth/', obtain_auth_token),  
]
```

7. Get Token:

Send username and password using POST request.

Example Response:

```
{  
    "token": "123abc456xyz"  
}
```

8. Use Token in Request Header:

Authorization: Token 123abc456xyz

Conclusion:

Token authentication in DRF provides secure user authentication by using tokens for API access instead of sending passwords repeatedly.

14. OpenWeatherMap API Integration

Q1) Introduction to OpenWeatherMap API and how to retrieve weather data.

Introduction to OpenWeather API:

OpenWeatherMap API is a weather service API that provides real-time weather information such as:

- Temperature
- Humidity
- Wind speed
- Weather conditions

Developers use this API in websites and mobile applications to show weather data.

Steps to Retrieve Weather Data:

1. Create an OpenWeatherMap Account:

- Sign up on the OpenWeather website
- Get your API key

2. Install Requests Package:

```
pip install requests
```

3. Python Example to Get Weather Data:

```
import requests
```

```
api_key = "your_api_key"  
city = "Surat"
```

```
url = f"https://api.openweathermap.org/data/2.5/weather?q={city}&appid={api_key}&units=metric"
```

```
response = requests.get(url)
```

```
data = response.json()
```

```
print("City:", data["name"])  
print("Temperature:", data["main"]["temp"]) print("Weather:",  
data["weather"][0]["description"]) Explanation:
```

- requests.get() sends a request to the API
- response.json() converts response into JSON format
- Weather data is retrieved from JSON

Sample Output:

```
City: Surat  
Temperature: 32  
Weather: clear sky  
Advantages:
```

- Real-time weather data
- Easy API integration
- JSON response format
- Useful for web and mobile apps

Conclusion:

OpenWeatherMap API helps developers retrieve live weather information easily using API requests and JSON responses.

15. Google Maps Geocoding API

Q1) Using Google Maps Geocoding API to convert addresses into coordinates.

Using Google Maps Geocoding API means converting an address into geographic coordinates like latitude and longitude.

This process is called Geocoding.

Example:

Address: Surat, Gujarat

Converted Coordinates:

Latitude: 21.1702

Longitude: 72.8311

Steps to Use Google Maps Geocoding API:

1. Get Google API Key:

- Create a Google Cloud account
- Enable Geocoding API
- Generate API key

2. Install Requests Package:

```
pip install requests
```

3. Python Example:

```
import requests
```

```
address = "Surat, Gujarat"
```

```
api_key = "your_api_key"
```

```
url = f"https://maps.googleapis.com/maps/api/geocode/json?address={address}&key={api_key}"
```

```
response = requests.get(url)
```

```
data = response.json()
```

```
location = data['results'][0]['geometry']['location']
```

```
print("Latitude:", location['lat']) print("Longitude:",  
location['lng'])
```

Explanation:

- requests.get() sends API request
- API returns JSON data
- Latitude and longitude are extracted from JSON response

Sample Output:

Latitude: 21.1702

Longitude: 72.8311 Uses:

- Map applications
- Location tracking

- Delivery systems
- Travel applications

Advantages:

- Accurate location data
- Easy integration
- Fast address conversion
- Useful for map services

Conclusion:

Google Maps Geocoding API helps developers convert addresses into latitude and longitude coordinates for location-based applications.

16. GitHub API Integration

Q1) Introduction to GitHub API and how to interact with repositories, pull requests, and issues.

Introduction to GitHub API:

GitHub API is a REST API that allows developers to interact with GitHub repositories, pull requests, issues, users, and other GitHub features programmatically.

Using GitHub API, developers can:

- Access repositories
 - Create pull requests
 - Manage issues
 - Fetch user information
 - Automate GitHub tasks
- Advantages:
- Automates GitHub operations
 - Easy repository management
 - Useful for DevOps and CI/CD
 - Supports JSON responses

1. Install Requests Package:

```
pip install requests
```

2. Access Repository Information:

Example:

```
import requests
```

```
url = "https://api.github.com/repos/python/cpython"
```

```
response = requests.get(url)

data = response.json()

print("Repository Name:", data["name"])
print("Owner:", data["owner"]["login"]) print("Stars:",
data["stargazers_count"])
```

Sample Output:

```
Repository Name: cpython
Owner: python Stars:
50000
```

3. Get Pull Requests:

Example:

```
import requests

url = "https://api.github.com/repos/python/cpython/pulls"

response = requests.get(url)

pull_requests = response.json()

for pr in pull_requests:
    print(pr["title"])
Explanation:
```

- /pulls endpoint retrieves pull requests
- Shows pull request titles

4. Get Issues:

Example:

```
import requests

url = "https://api.github.com/repos/python/cpython/issues"

response = requests.get(url)

issues = response.json()

for issue in issues:
    print(issue["title"])
```

Explanation:

- /issues endpoint retrieves repository issues

- Displays issue titles

5. Authentication with Token:

Example:

```
headers = {
    "Authorization": "token your_github_token"
}
```

response = requests.get(url, headers=headers) Importance:

- Used in automation
- Helps manage repositories
- Useful in software development workflows
- Integrates with applications easily

Conclusion:

GitHub API helps developers interact with repositories, pull requests, and issues programmatically using REST API requests and JSON responses.

17. Twitter API Integration

Q1) Using Twitter API to fetch and post tweets, and retrieve user data.

Using X API means interacting with Twitter services programmatically to fetch tweets, post tweets, and retrieve user information. Twitter API helps developers:

- Read tweets
- Post tweets
- Get user profile data
- Build social media applications

Advantages:

- Real-time social media data
- Easy automation
- Useful for analytics and bots
- API returns JSON data

1. Install Tweepy Package:

pip install tweepy

2. Import Tweepy:

```
import tweepy
```

3. Authentication:

```
api_key = "your_api_key" api_secret  
= "your_api_secret" access_token =  
"your_access_token"  
access_secret = "your_access_secret"
```

```
auth = tweepy.OAuth1UserHandler(  
api_key, api_secret,  
access_token, access_secret  
)
```

```
api = tweepy.API(auth)
```

4. Fetch Tweets:

Example:

```
tweets = api.home_timeline()
```

```
for tweet in tweets:  
print(tweet.text)
```

Explanation:

- Retrieves recent tweets from timeline
- Displays tweet text

5. Post Tweet:

Example:

```
api.update_status("Hello from Python API")
```

Explanation:

- Posts a new tweet to the account

6. Retrieve User Data:

Example:

```
user = api.get_user(screen_name="Twitter")
```

```
print("Name:", user.name) print("Followers:",  
user.followers_count)
```

Sample Output:

Name: Twitter Followers:

60000000 Uses:

- Social media applications
- Tweet automation
- Analytics systems
- Chatbots

Conclusion:

Twitter API helps developers fetch tweets, post tweets, and retrieve user information easily using API requests and JSON responses.

18. REST Countries API Integration

Q1) Introduction to REST Countries API and how to retrieve country-specific data.

REST Countries API is a free REST API that provides information about countries such as:

- Country name
- Capital
- Population
- Currency
- Region
- Flag

Developers use this API to retrieve country-specific data in web and mobile applications.

Advantages:

- Free to use
- Easy API integration
- JSON response format
- Useful for country information systems

1. Install Requests Package:

```
pip install requests
```

2. Python Example to Retrieve Country Data:

Example:

```
import requests
```

```
country = "India"
```

```
url = f"https://restcountries.com/v3.1/name/{country}"
```

```
response = requests.get(url)
```

```
data = response.json()
```

```
print("Country:", data[0]["name"]["common"])
print("Capital:", data[0]["capital"][0]) print("Region:",
data[0]["region"]) print("Population:",
data[0]["population"]) Explanation:
```

- requests.get() sends API request
- response.json() converts response into JSON
- Country data is extracted from JSON response

Sample Output:

```
Country: India
Capital: New Delhi
Region: Asia
Population: 1400000000
```

3. Retrieve Currency Information:

Example:

```
print(data[0]["currencies"])
```

4. Retrieve Country Flag:

Example:

```
print(data[0]["flags"]["png"])
```

Uses:

- Travel applications
- Country information systems
- Educational websites
- Geography applications

Conclusion:

REST Countries API helps developers easily retrieve country-specific information using REST API requests and JSON responses.

19. Email Sending APIs (SendGrid, Mailchimp)

Q1) Using email sending APIs like SendGrid and Mailchimp to send transactional emails.

Using email sending APIs like SendGrid and Mailchimp means sending transactional emails from applications using third-party email services.

Transactional emails are automatic emails sent to users for:

- OTP verification
- Password reset
- Order confirmation
- Account registration
- Notifications

Advantages:

- Fast email delivery
- Reliable email service
- Easy API integration
- Better email management

1. Using SendGrid API:

Install SendGrid:

```
pip install sendgrid
```

Example:

```
from sendgrid import SendGridAPIClient
from sendgrid.helpers.mail import Mail
```

```
message = Mail(
    from_email='test@example.com',
    to_emails='user@example.com',    subject='Welcome
Email',
    html_content='<strong>Welcome to our website</strong>'
)
```

```
sg = SendGridAPIClient('your_api_key')
```

```
response = sg.send(message)
```

```
print(response.status_code) Explanation:
```

- Mail() creates email message
- SendGridAPIClient() connects with SendGrid API
- sg.send() sends the email

2. Using Mailchimp API:

Install Mailchimp Package:

```
pip install mailchimp-marketing
```

Example:

```
from mailchimp_marketing import Client
```

```
client = Client()

client.set_config({
    "api_key": "your_api_key",
    "server": "your_server_prefix"
})

response = client.ping.get()
```

print(response) Explanation:

- Connects application with Mailchimp API
- Used for email campaigns and transactional emails

Uses:

- User verification
- Password recovery
- Order notifications
- Marketing emails

Conclusion:

SendGrid and Mailchimp APIs help applications send transactional emails quickly, securely, and efficiently.

20. SMS Sending APIs (Twilio)

Q1) Introduction to Twilio API for sending SMS and OTPs.

Introduction to Twilio API:

Twilio API is a cloud communication API used to send:

- SMS messages
- OTPs (One-Time Passwords)
- Voice calls
- Notifications

Developers use Twilio API in web and mobile applications for user verification and communication.

Advantages:

- Fast SMS delivery
- Secure OTP verification
- Easy API integration
- Supports global messaging

Steps to Use Twilio API:

1. Create Twilio Account:

- Sign up on Twilio
- Get Account SID and Auth Token
- Get Twilio phone number

2. Install Twilio Package:

```
pip install twilio
```

3. Python Example to Send SMS:

```
from twilio.rest import Client
```

```
account_sid = "your_account_sid"
auth_token = "your_auth_token"
```

```
client = Client(account_sid, auth_token)
```

```
message = client.messages.create(
    body="Hello from Twilio API",
    from_="+1234567890",
    to="+919999999999"
)
```

```
print(message.sid) Explanation:
```

- Client() connects with Twilio API
- messages.create() sends SMS
- body contains message text
- from_ is Twilio number
- to is receiver number

4. Sending OTP Example:

```
otp = "1234"
```

```
message = client.messages.create(
    body=f"Your OTP is {otp}",
    from_="+1234567890",
    to="+919999999999"
)
```

Uses:

- User registration
- Login verification
- Two-factor authentication
- Password reset systems

Conclusion:

Twilio API helps developers send SMS messages and OTPs securely and efficiently in web and mobile applications.

21. Payment Integration (PayPal, Stripe)

Q1) Introduction to integrating payment gateways like PayPal and Stripe.

Introduction to integrating payment gateways like PayPal and Stripe:

Payment gateway integration means connecting online payment services with websites or applications to accept online payments securely.

Payment gateways are used for:

- Online shopping websites
- Subscription systems
- Ticket booking
- Mobile applications

Advantages:

- Secure online payments
- Fast transaction processing
- Supports cards and digital payments
- Improves user experience

1. PayPal Integration:

Install PayPal Package:

```
pip install paypalrestsdk
```

Example:

```
import paypalrestsdk
```

```
paypalrestsdk.configure({  
    "mode": "sandbox",  
    "client_id": "your_client_id",  
    "client_secret": "your_client_secret"  
})
```

```
payment = paypalrestsdk.Payment({  
    "intent": "sale",  
    "payer": {  
        "payment_method": "paypal"  
    },  
    "transactions": [{  
        "amount": {  
            "total": "10.00",  
            "currency": "USD"  
        },  
        "description": "Payment Description"  
    }],  
    "redirect_urls": {
```

```

        "return_url": "http://localhost:8000/success",
        "cancel_url": "http://localhost:8000/cancel"
    }
})

```

```

if payment.create():
    print("Payment Created")
else:
    print(payment.error)

```

2. Stripe Integration:

Install Stripe Package:

```
pip install stripe
```

Example:

```
import stripe
```

```
stripe.api_key = "your_secret_key"
```

```

payment_intent = stripe.PaymentIntent.create(
    amount=5000,    currency="usd",
    payment_method_types=["card"],
)

```

```
print(payment_intent) Explanation:
```

- PaymentIntent.create() creates payment request
- Amount is in cents
- Stripe handles secure card payments

Uses:

- E-commerce websites
- Subscription platforms
- Donation systems
- Online services

Conclusion:

PayPal and Stripe payment gateways help developers securely process online payments in web and mobile applications.

22. Google Maps API Integration

Q1) Using Google Maps API to display maps and calculate distances between locations.

Using Google Maps API means displaying maps in websites or applications and calculating distances between different locations.

Google Maps API helps developers:

- Show maps
- Find locations
- Calculate distances
- Display routes and directions

Advantages:

- Accurate map data
- Real-time location services
- Easy API integration
- Useful for travel and delivery apps

1. Get Google Maps API Key:

- Create Google Cloud account
- Enable Maps JavaScript API and Distance Matrix API • [Generate API key](#)

2. Display Google Map:

HTML Example:

```
<!DOCTYPE html>
<html>
<head>
  <title>Google Map</title>
</head>
<body>

<div id="map" style="height:500px;width:100%;"></div>

<script>
function initMap() {

  var location = {
lat: 21.1702,
lng: 72.8311
  };

  var map = new google.maps.Map(document.getElementById("map"), {
zoom: 10,      center: location
  });

  var marker = new google.maps.Marker({
position: location,      map: map
  });
}
</script>
```

```
<script src="https://maps.googleapis.com/maps/api/js?key=YOUR_API_KEY&callback=initMap" async defer></script>

</body> </html>
```

Explanation:

- google.maps.Map() displays map
- Marker() adds location marker
- Latitude and longitude set map position

3. Calculate Distance Between Locations:

Python Example:

```
import requests

api_key = "your_api_key"

origin = "Surat"
destination = "Ahmedabad"

url =
f"https://maps.googleapis.com/maps/api/distancematrix/json?origins={origin}&destinations={destination}&key={api_key}"
response = requests.get(url)

data = response.json()

distance = data['rows'][0]['elements'][0]['distance']['text']

print("Distance:", distance)
```

Sample Output:

Distance: 265 km Uses:

- Delivery applications
- Cab booking systems
- Travel websites
- Navigation systems

Conclusion:

Google Maps API helps developers display maps, calculate distances, and provide location services in web and mobile applications.